Modelos locales con portátiles Ubuntu

Manual del prototipo de dos equipos

Proyecto docente de iesataulfoargentasor

Ubuntu 22.04 LTS · llama.cpp · CUDA · RPC
Qwen3-14B Q4_K_M · Ethernet Gigabit

Edición de consulta sin Internet · 23 de octubre de 2026

Documentación basada en la revisión f962ba4 del repositorio. Incluye el contexto, el hardware observado, los seis esquemas, todos los pasos de laboratorio y la vuelta a DHCP.

Estado: preparación y pruebas simuladas completadas. La ejecución distribuida en los portátiles del centro sigue pendiente.

Los esquemas están incorporados al archivo. Los enlaces internos llevan a capítulos del manual; las referencias externas necesitan Internet. Los scripts y los pesos se obtienen por separado antes de la sesión aislada.

Repositorio: iesataulfoargentasor/cluster-llm-portatiles

Índice

1. Clúster de modelos locales con portátiles Ubuntu 3

2. Origen y evolución del proyecto 7

3. Hardware identificado y requisitos del prototipo 9

4. Arquitectura y decisiones del prototipo 11

5. Orden de ejecución en A y B 14

6. Prueba de Qwen3 entre dos portátiles Ubuntu 18

7. Validación y registro de resultados 26

8. Adaptación a otros equipos y ampliación a cuatro 28

9. Fuentes y versiones 31

10. Comprobaciones del paquete 32

11. Recopilación de datos de hardware 33

12. Versiones y huella del modelo 34

Para la práctica, comienza por «Orden de ejecución en A y B». La guía completa contiene requisitos, resultados esperados y resolución de problemas. Ese capítulo recoge también el contenido operativo de la hoja resumen imprimible.

1. Clúster de modelos locales con portátiles Ubuntu

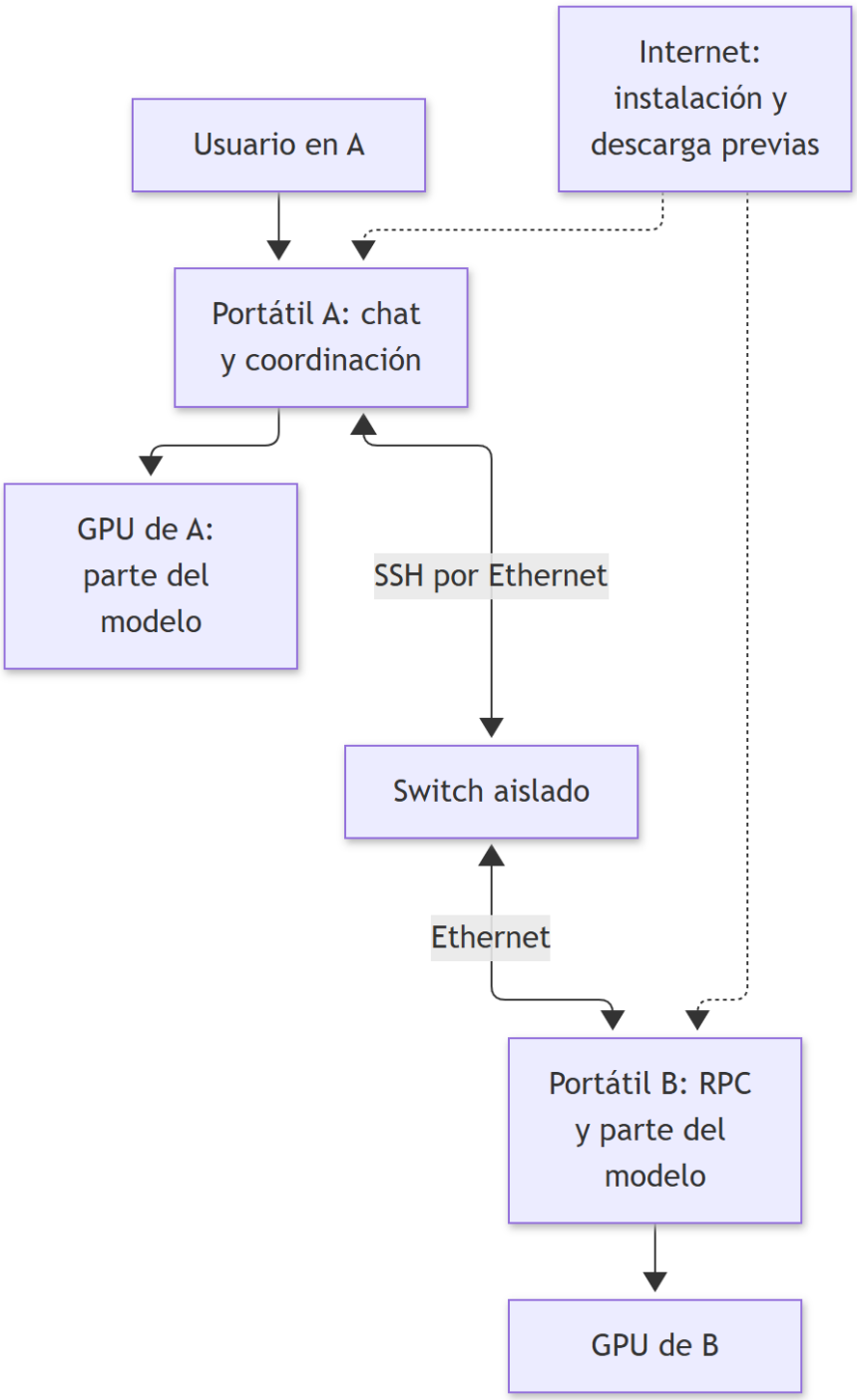
Proyecto docente de **iesataulfoargentasor** para estudiar si los portátiles disponibles en el centro pueden colaborar ejecutando un mismo modelo de lenguaje local. El primer objetivo es construir un prototipo con **dos equipos**; después se decidirá, a partir de mediciones, si merece la pena ampliarlo a **cuatro**.

La necesidad inicial era trabajar con modelos de pesos abiertos sin depender de un servicio de inferencia en Internet, aprovechando equipos existentes. La dificultad era la memoria de cada GPU. La solución preparada usa **llama.cpp con CUDA y RPC**, con un coordinador A y una GPU remota en B, conectados por Ethernet.

Estado del proyecto

Preparación del prototipo, a 23 de octubre de 2026. Se ha caracterizado un portátil, preparado scripts e instrucciones y superado 12 pruebas de lógica de red simulada. Todavía falta instalar y ejecutar el conjunto en los dos portátiles del centro. No hay medidas reales de tokens por segundo ni confirmación de que Qwen3-14B quepa con la configuración propuesta. Los tres informes recibidos proceden del mismo equipo.

El objetivo en un dibujo



La prueba distribuye trabajo y memoria entre GPU. No convierte físicamente las dos tarjetas en una sola: hay comunicación por red y dependencias entre cálculos. Más equipos pueden aumentar la capacidad, pero la velocidad se debe medir.

Por dónde empezar

Manual para consultar sin Internet: descargar el PDF completo · ver el PDF en GitHub.

Incluye 34 páginas, índice enlazado, 12 capítulos y los seis esquemas incorporados. Guárdalo en ambos portátiles antes de aislar la red. Edición del 23 de octubre de 2026, basada en la revisión documental `f962ba4`. El PDF contiene las instrucciones; los scripts y el modelo se preparan por separado según la guía. Las referencias externas requieren Internet.

Necesidad	Documento
Entender cómo surgió y evolucionó	Historia del proyecto
Conocer el equipo real y sus límites	Hardware identificado
Entender A, B, CUDA, RPC y la red	Arquitectura y decisiones
Ejecutar el prototipo Ubuntu 22.04	Guía completa
Consultar los comandos en orden	Resumen para GitHub
Llevar una hoja al laboratorio	Hoja HTML imprimible, abrir localmente tras descargar
Registrar lo que realmente ocurre	Validación y ficha de resultados
Preparar otros equipos o cuatro nodos	Adaptación y ampliación
Entender las comprobaciones del paquete	Registro de comprobaciones

Obtener los archivos en Ubuntu

Con Internet, en cada portátil, descarga el repositorio mediante **Code → Download ZIP**. En Archivos de Ubuntu, abre Descargas, pulsa con el botón derecho sobre el ZIP y selecciona **Extraer aquí**. Abre la carpeta extraída y copia su carpeta `prueba_dos_portatiles` a tu Carpeta personal.

Como alternativa, si tienes Git instalado, abre una terminal y ejecuta:

```
cd "$HOME"
git clone https://github.com/iesataulfoargentasor/cluster-llm-portatiles.git
```

Copia la carpeta `prueba_dos_portatiles` del repositorio a tu Carpeta personal. La guía supone que queda en `~/prueba_dos_portatiles`. Si ya tienes una copia de una sesión anterior, consérvala con otro nombre antes de copiar esta. Mantén la misma versión del paquete en A y B.

Abre una terminal con **Ctrl+Alt+T** y comprueba que los archivos están donde corresponde:

```
cd "$HOME/prueba_dos_portatiles"
ls
```

Debes ver los scripts 01 a 08, `comun.sh`, `versiones.env`, `red_cluster.py` y `probar_modelo.py`. A continuación sigue el orden de ejecución en A y B. Cada bloque indica en qué equipo y terminal ejecutarlo; al abrir una terminal nueva, vuelve a entrar en la carpeta del paquete.

Los documentos y scripts se descargan del repositorio; **los pesos del modelo y CUDA se descargan después**, durante los pasos de preparación. Guarda la guía HTML y la hoja resumen localmente antes de desconectar Internet. GitHub muestra el código de los HTML: para leerlos con formato, descárgalos y ábrelos en el navegador.

Configuración concreta del prototipo

- Ubuntu **22.04.5 LTS**, Intel i7-12700H, RAM nominal 32 GB.

- NVIDIA **RTX 3060 Laptop, 6144 MiB de VRAM**, controlador observado 580.173.02.

- Ethernet **1 Gb/s**, interfaz observada `eno1`.

- CUDA Toolkit 12.8; compilación para capacidad CUDA 8.6.

- Qwen3-14B Q4_K_M; contexto 4096, una conversación, reparto por capas 1:1.

- IP de A: `192.168.50.10/24`; IP de B: `192.168.50.20/24`.

- Misma revisión de llama.cpp en ambos. Versiones, tamaño y SHA-256 en versiones.env.

Contenido y datos del laboratorio

El repositorio publica instrucciones, scripts y una síntesis técnica. Los informes brutos, direcciones MAC, identificadores de máquinas, consultas y registros reales permanecen fuera del repositorio. Tampoco contiene modelos, binarios de CUDA ni el artículo Word aportado como referencia durante la conversación. Antes de compartir un resultado, elimina datos personales o de la red del centro.

Los scripts de red se ejecutan localmente en Ubuntu y conservan los perfiles previos. RPC se mantiene en localhost y se transporta por SSH. La implementación oficial considera RPC experimental; el alcance es un laboratorio controlado. Véanse las fuentes y versiones.

La documentación y los scripts se han preparado con asistencia de IA y revisión local. Los resultados de laboratorio deben completar esta primera edición. No se ha asignado una licencia general de redistribución a este repositorio; los componentes externos conservan sus propias licencias.

2. Origen y evolución del proyecto

La necesidad docente

El proyecto nace de la necesidad de ejecutar modelos de lenguaje en local utilizando los portátiles disponibles en el centro. La experiencia previa con herramientas como LM Studio y llama.cpp situaba el trabajo en una única máquina: el modelo debía ajustarse a su memoria o recurrir parcialmente a RAM y CPU.

La primera pregunta fue si cuatro portátiles conectados a un switch podrían colaborar para ejecutar un modelo mayor. La idea inicial hablaba de procesadores i7, GPU de 8 GB y Ubuntu 24.04. Esas características eran una estimación que había que comprobar antes de elegir software y modelo.

De la pregunta al diagnóstico

Se solicitaron comandos para identificar GPU, RAM y Ethernet. Después se preparó un script de recopilación, porque el portátil personal desde el que se organizaba el trabajo no era uno de los equipos del centro. Así se pudieron recoger datos directamente en el equipo destinado a la prueba.

También se aportó un artículo en Word sobre ejecutar un modelo entre varias máquinas como material de referencia. Su existencia ayudó a situar la investigación; no se incorporan aquí su texto ni supuestos resultados como evidencia de funcionamiento de estos portátiles.

Se recibieron tres informes del mismo portátil. El diagnóstico corrigió dos supuestos importantes: el sistema era **Ubuntu 22.04.5 LTS** y la GPU tenía **6 GiB**, no 8 GB. La Ethernet negociaba a **1000 Mb/s Full Duplex**. Esos datos determinaron el diseño del primer prototipo.

Por qué empezar con dos

Se decidió reducir el primer montaje a A y B para identificar por separado problemas de instalación, memoria, red y reparto del modelo. El objetivo de esta fase es comprobar que las dos GPU participan, que el modelo carga y que se pueden obtener mediciones repetibles. Añadir dos nodos más antes de conseguirlo multiplicaría las variables.

Se preparó una instalación con llama.cpp, CUDA y RPC, un modelo Qwen3-14B Q4_K_M y un túnel SSH. A mantiene el archivo del modelo y la interfaz de chat; B aporta su GPU. Se fijaron versiones para que ambos compilen el mismo código.

El laboratorio sin Internet

La siguiente cuestión fue si la prueba podía funcionar en un switch aislado. Se separaron dos fases: instalación y descarga con Internet; inferencia posterior entre los equipos sin Internet. Como un switch aislado normalmente no asigna direcciones, se añadieron scripts para IP fija y retorno a DHCP.

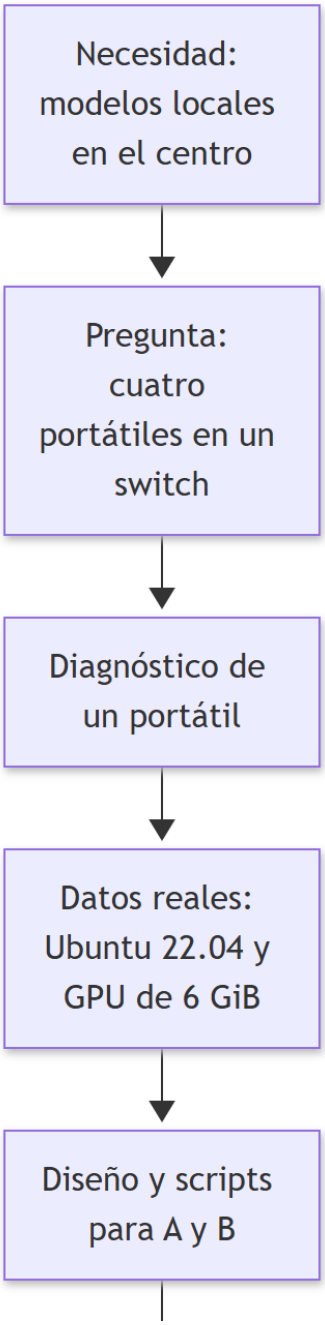
Los scripts crean un perfil Ethernet de laboratorio y conservan los perfiles habituales. La vuelta a DHCP se diseñó para indicar claramente cuándo falta un servidor que entregue una dirección. Se incorporó una hoja resumen que distingue las terminales que deben quedar abiertas.

Qué pruebas se han realizado

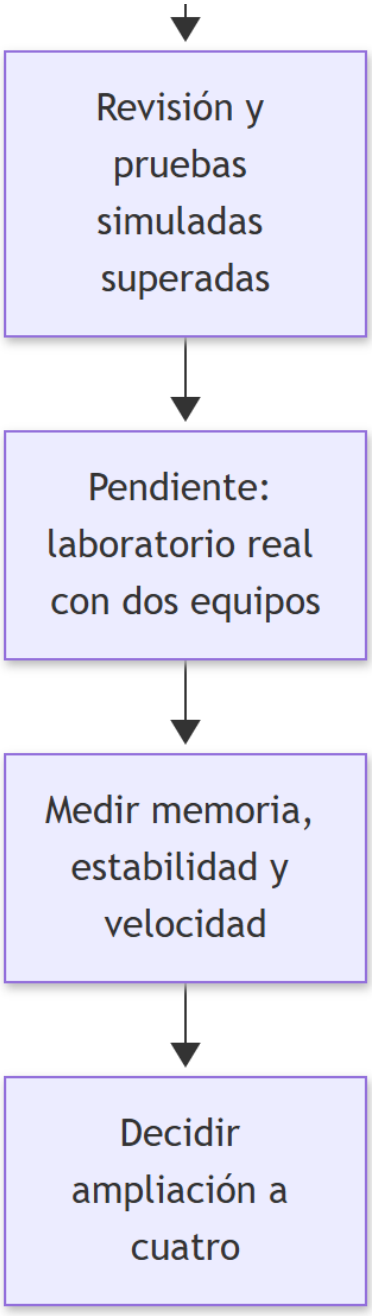
El trabajo efectuado en el portátil personal comprende comprobaciones de sintaxis y simulaciones de lógica. La revisión de red superó 12 casos simulados y llevó a corregir avisos de enlace, recuperación ante errores de entrada/salida y detección de perfiles alterados. Durante estas pruebas no se cambió la red del portátil personal.

La recopilación de hardware sí se ejecutó en un portátil del centro. **No equivale a haber ejecutado el modelo distribuido.** Esa prueba, el informe del segundo equipo y la medición del rendimiento siguen pendientes.

1. Inicio del recorrido



2. Continuación



Esta es una reconstrucción del proceso a partir de las peticiones y los archivos conservados, no una transcripción literal de la conversación. Los informes de hardware utilizados están fechados el 23 de octubre de 2026.

3. Hardware identificado y requisitos del prototipo

Datos comprobados en un equipo

Síntesis de tres informes del mismo portátil, usando el último para el detalle de Ethernet. No representan tres nodos distintos.

Elemento	Dato observado	Consecuencia
Sistema	Ubuntu 22.04.5 LTS	El instalador exige Ubuntu 22.04 x86_64
CPU	Intel Core i7-12700H	Coordinación y tareas auxiliares
RAM	32518660 kB de MemTotal, unos 31 GiB utilizables	Equipo de RAM nominal 32 GB
GPU	NVIDIA RTX 3060 Laptop	Backend CUDA
VRAM	6144 MiB, 6 GiB	Presupuesto de memoria limitado
Controlador	580.173.02	Se conserva el controlador existente
Ethernet	1000 Mb/s, Full Duplex	Límite práctico de la comunicación entre nodos
Interfaz	eno1	Valor por defecto en los scripts de red
Modos del adaptador	10, 100 y 1000 Mb/s	Un switch más rápido por sí solo no aumenta su enlace

Confirmar el segundo equipo

Conecta Ethernet al switch y ejecuta, desde la carpeta `herramientas` del repositorio:

```
bash ./recopilar_datos_cluster.sh
```

El informe queda en `~/informes_cluster`. Si falta `ethtool`, instálalo con Internet y repite la recopilación, como indica el script. Comprueba además `nvidia-smi` en los dos equipos antes de instalar. Los informes contienen nombres de equipo y datos de interfaces: revísalos antes de publicarlos.

Memoria y espacio

Dos GPU iguales sumarían 12 GiB de VRAM nominal, distribuida entre dos dispositivos. El GGUF elegido ocupa 9.001.752.960 bytes en disco. Queda por comprobar el margen para caché de contexto, buffers, escritorio y otros procesos; el tamaño del archivo no representa todo el consumo de ejecución.

Los scripts requieren al menos 5000 MiB de VRAM libre por equipo antes del arranque. El contexto inicial es 4096; la guía ofrece 2048 si falta memoria. No se presupone que cuatro GPU de 6 GiB permitan cualquier modelo de 24 GiB: también necesitan memoria adicional y comunicación.

Reserva preferiblemente 40 GB de disco libres por portátil; el instalador exige al menos 30 GiB. A guarda el GGUF y B puede conservar caché de tensores. Mantén cargadores conectados y evita suspensión durante la sesión.

Identificación local

Anota para cada nodo su etiqueta A o B, usuario de Ubuntu, interfaz Ethernet y espacio libre. No copies el estado `~/cluster_llm/red` entre máquinas. El segundo portátil debe documentarse antes de tratarlo como idéntico al primero.

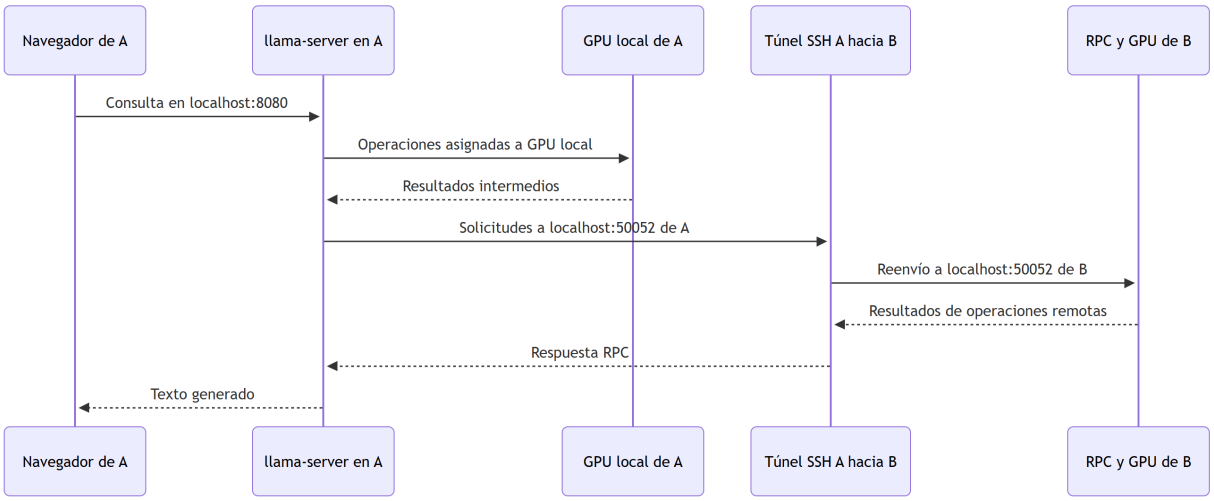
4. Arquitectura y decisiones del prototipo

Función de cada equipo

Equipo	Archivos y procesos	Terminales durante la prueba
A	GGUF completo, llama-server, GPU local, túnel SSH y navegador	A1 túnel, A2 modelo, A3 consulta de medición
B	ggml-rpc-server, GPU remota y posible caché de tensores	B1 RPC

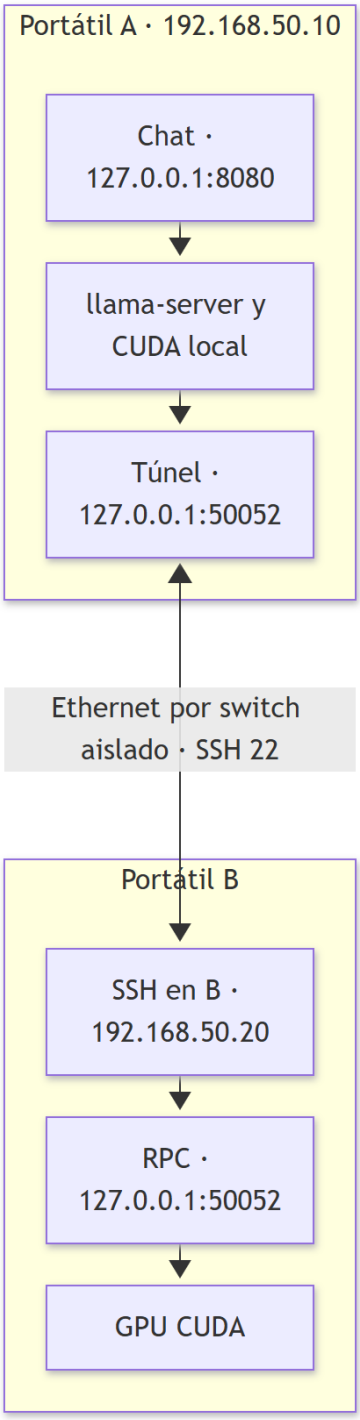
CUDA permite utilizar la GPU NVIDIA. RPC permite que llama.cpp solicite operaciones a dispositivos remotos. El modelo utiliza reparto por capas: los cálculos tienen dependencias, por lo que no hay que esperar que ambas GPU estén al 100 % a la vez.

Recorrido de una consulta



Esquema conceptual: el proceso repite operaciones durante la generación. La transferencia inicial de pesos se produce al cargar el modelo; no se vuelve a transferir todo el GGUF por cada token.

Direcciones y puertos



`127.0.0.1` siempre corresponde al propio equipo. La página del chat se abre en A. RPC no se publica directamente en la red; el túnel llega a su puerto local en B. SSH añade trabajo y sus efectos forman parte de las mediciones de este montaje.

Decisiones adoptadas

- **Dos equipos inicialmente:** permite aislar problemas antes de ampliar.

- **llama.cpp y GGUF:** formato y herramientas utilizados por el paquete. Este montaje no configura LM Studio.

- **Qwen3-14B Q4_K_M:** candidato de prueba cuantizado; su viabilidad se confirmará al cargarlo.

- **Versiones fijadas:** permiten repetir instalación y comparar A y B.

- **Contexto 4096 y una conversación:** limita el consumo inicial y facilita interpretar medidas.

- **Reparto por capas 1:1 y ajuste automático desactivado:** facilita observar la configuración solicitada. Si no cabe, se revisa explícitamente.

- **Instalación previa con Internet:** el funcionamiento aislado depende de disponer ya de herramientas, binarios, pesos y documentos.

- **Perfiles Ethernet separados:** permiten aplicar IP de laboratorio y recuperar DHCP sin sobrescribir el perfil habitual.

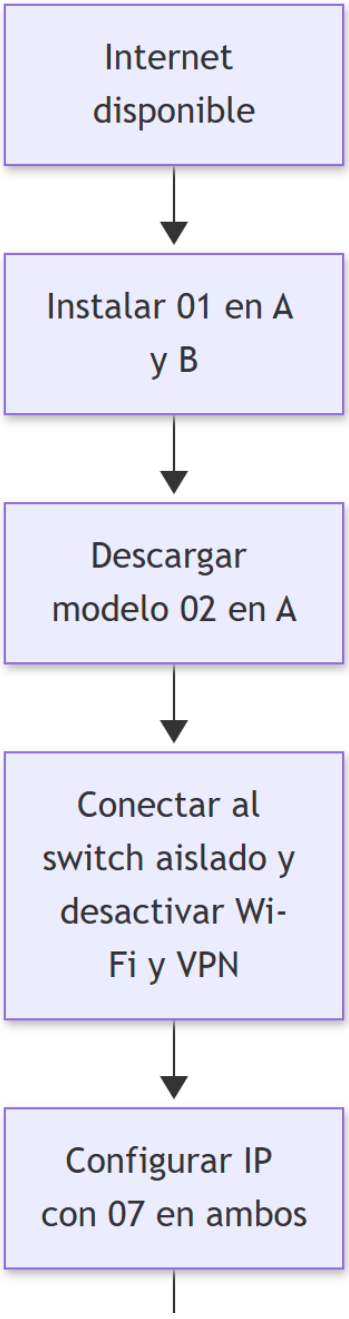
La documentación oficial describe RPC como experimental y permite uno o varios servidores remotos. Esa posibilidad técnica no acredita rendimiento en esta Ethernet. Referencia de RPC en la revisión fijada.

5. Orden de ejecución en A y B

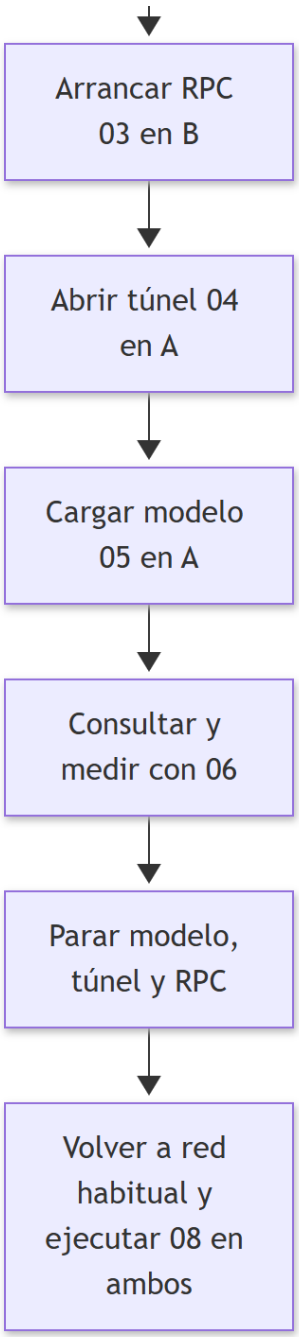
Sigue esta secuencia con el paquete completo en `~/prueba_dos_portatiles`. Utiliza una terminal local y el mismo usuario en cada equipo. No antepongas `sudo` a los scripts. Si un paso falla, consulta la guía completa antes de continuar.

Si empiezas desde cero, completa primero la descarga y colocación de archivos. Esta secuencia se realiza en los portátiles Ubuntu del proyecto. Cada vez que abras una terminal nueva, ejecuta `cd "$HOME/prueba_dos_portatiles"` antes de un script.

1. Inicio del recorrido



2. Continuación



1 Preparación con Internet

En ambos, comprueba primero la GPU y la interfaz Ethernet:

```
cd "$HOME/prueba_dos_portatiles"
nvidia-smi
nmcli device status
```

Conecta cargadores, evita suspensión y reserva espacio. Este montaje presupone el hardware descrito en la ficha técnica.

En A:

```
bash ./01_instalar.sh A
```

Ejecuta 02 solo después de que 01 termine correctamente. Espera a la comprobación de tamaño y hash del modelo.

```
bash ./02_descargar_modelo_A.sh
```

En B:

```
bash ./01_instalar.sh B
whoami
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Anota el usuario y la huella de B. Las instalaciones de A y B pueden realizarse al mismo tiempo. Termina ambas y la descarga antes de desconectar Internet.

2 Switch aislado

Conecta A y B al switch encendido y separado de la red del centro. Desactiva Wi-Fi y VPN desde Ubuntu para mantener el aislamiento.

En A:

```
bash ./07_configurar_red_aislada.sh A
```

En B:

```
bash ./07_configurar_red_aislada.sh B
```

Estos comandos usan `eno1`. Si la interfaz cambia, añádela: `bash ./07_configurar_red_aislada.sh B enp3s0`. Usa después esa misma interfaz al restaurar DHCP.

En A, cuando ambos tengan IP:

```
ping -c 3 192.168.50.20
```

A tiene `192.168.50.10/24` y B `192.168.50.20/24`, sin puerta de enlace ni DNS. Si ping no responde, revisa cable, IP y cortafuegos; el filtrado de ping no implica necesariamente que SSH falle.

3 Arrancar el conjunto

B, terminal B1:

```
cd "$HOME/prueba_dos_portatiles"
sudo systemctl start ssh
bash ./03_iniciar_rpc_B.sh
```

Espera a que aparezcan CUDA0 y el endpoint local. Deja B1 abierta.

A, terminal A1: sustituye `USUARIO_B` por el usuario anotado en B.

```
cd "$HOME/prueba_dos_portatiles"
```

```
bash ./04_abrir_tunel_A.sh USUARIO_B@192.168.50.20
```

Compara la huella SSH con la anotada y acepta si coincide. Introduce la contraseña de B cuando se solicite. Deja A1 abierta, aunque ya no muestre texto.

A, nueva terminal A2:

```
cd "$HOME/prueba_dos_portatiles"
bash ./05_iniciar_modelo_A.sh
```

Espera a que el servidor esté listo y deja A2 abierta. Abre en el navegador de A `http://127.0.0.1:8080` y realiza una consulta breve.

4 Observar y medir

En una terminal adicional de cada equipo:

```
watch -n 1 nvidia-smi
```

Comprueba VRAM ocupada en ambas GPU y los registros de carga. No es necesario ver actividad máxima simultánea. Ctrl+C cierra la observación.

En A, nueva terminal A3, con el modelo listo:

```
cd "$HOME/prueba_dos_portatiles"
bash ./06_probar_A.sh
```

No uses el chat mientras se mide. Repite el comando tres veces si quieres comparar consultas consecutivas. Completa la ficha de resultados; el tiempo de carga se mide por separado.

5 Finalizar y recuperar DHCP

Pulsa Ctrl+C en este orden:

1. A2: servidor del modelo.

2. A1: túnel SSH.

3. B1: servidor RPC.

Conecta Ethernet a la red habitual con DHCP. En una terminal local de cada portátil:

```
cd "$HOME/prueba_dos_portatiles"
bash ./08_restaurar_dhcp.sh
```

Si usaste otra interfaz: `bash ./08_restaurar_dhcp.sh enp3s0`. Espera el mensaje de conexión DHCP activa. Si aún estás en el switch aislado, puede quedar pendiente: conecta a la red habitual y repite 08. Conserva `~/cluster_llm/red`. Restaura Wi-Fi, VPN y suspensión según sus ajustes habituales.

Si el perfil anterior tenía IP manual, 08 crea un perfil DHCP separado y conserva el original. En una red del centro con configuración especial, selecciona su perfil habitual desde Ubuntu.

Si SSH está bloqueado por UFW

En otra terminal de B, consulta `sudo ufw status`. Solo si está activo y bloquea la conexión desde A:

```
sudo ufw allow from 192.168.50.10 to any port 22 proto tcp
```

Si añadiste esa regla únicamente para la prueba, retírala al terminar:

```
sudo ufw delete allow from 192.168.50.10 to any port 22 proto tcp
```

RPC permanece en localhost; no abras el puerto 50052 a la red. El servicio SSH puede seguir activo tras la prueba. Si se instaló solo para este proyecto y nadie más lo necesita, la guía explica cómo pararlo.

Otra sesión

Conserva instalación y modelo. Empieza de nuevo por la conexión al switch y 07; después 03 en B, 04 y 05 en A y finalmente 06. La numeración de scripts se mantiene por compatibilidad con el paquete original.

6. Prueba de Qwen3 entre dos portátiles Ubuntu

Esta guía permite repartir Qwen3-14B Q4_K_M entre dos portátiles con Ubuntu 22.04, NVIDIA RTX 3060 Laptop de 6 GB, 32 GB de RAM y Ethernet Gigabit. El portátil **A coordina el modelo y muestra el chat**. El portátil **B aporta su GPU**. Ambos calculan partes del mismo modelo.

La configuración inicial utiliza 4096 tokens de contexto, una conversación, caché de contexto Q8 y reparto por capas. La capacidad prevista debe confirmarse en los equipos: todavía no hay una medición real de velocidad ni una ejecución de este paquete en esos portátiles.

1. Qué necesitas y qué instala el paquete

- Dos portátiles con la configuración anterior. Solo tenemos un informe de hardware; comprueba también el segundo con `nvidia-smi` antes de instalar.

- Usuario normal con permisos de `sudo` en cada equipo. Los nombres de usuario pueden ser distintos.

- Cargadores conectados, tapas abiertas y suspensión automática desactivada temporalmente durante la prueba desde Configuración → Energía. Restaura después tu preferencia habitual.

- Cable Ethernet desde cada portátil al switch. La prueba usa la dirección cableada de B.

- Internet durante la instalación y la descarga inicial. Después se puede trabajar sin Internet.

- Recomendación de 40 GB libres en cada equipo para herramientas, compilación, modelo o caché. El instalador pide al menos 30 GiB libres al empezar.

Los scripts guardan sus archivos en `~/cluster_llm`. Instalan herramientas de compilación y, si falta, CUDA Toolkit 12.8 desde el repositorio oficial NVIDIA. Utilizan el controlador NVIDIA que ya existe; el equipo informado tiene 580.173.02. B incorpora el servidor OpenSSH. La instalación de paquetes puede pedir la contraseña de sudo.

Ejecuta los scripts con `bash`, como usuario normal. No pongas `sudo` delante del script completo. Los pasos que necesitan privilegios ya lo solicitan internamente.

Versiones fijadas: llama.cpp v0.7.0, revisión `620d2225cc436ab0b001e3200e590e3b286ef9ab`; CUDA Toolkit 12.8; modelo oficial Qwen3-14B Q4_K_M, revisión `530227a7d994db8eca5ab5ced2fb692b614357fd`. `versiones.env` contiene también el tamaño y la huella SHA-256 del modelo. Copia el mismo paquete a los dos equipos.

2. Copiar el paquete e identificar A y B

Descomprime el ZIP. Copia la carpeta completa `prueba_dos_portatiles` a la Carpeta personal de cada portátil. Deben quedar juntos los archivos numerados del 01 al 08, `comun.sh`, `versiones.env`, `probar_modelo.py` y `red_cluster.py`.

En todas las terminales que abras para estos pasos, entra primero en esa carpeta:

```
cd "$HOME/prueba_dos_portatiles"
```

Anota qué portátil será A y cuál será B. Puedes poner una etiqueta física. Mantén el mismo usuario de Ubuntu durante toda la prueba en cada portátil: los archivos se guardan en su carpeta personal.

Equipo	Función	Qué mantendrás abierto durante la prueba
A coordinador	GPU local, modelo y chat	Terminal del túnel, terminal del modelo y navegador
B remoto	GPU remota	Terminal de RPC

3. Instalar en ambos equipos

En A, con Internet:

```
cd "$HOME/prueba_dos_portatiles"
bash ./01_instalar.sh A
```

En B, con Internet:

```
cd "$HOME/prueba_dos_portatiles"
bash ./01_instalar.sh B
```

Puedes instalar en ambos a la vez. La descarga del toolkit y la compilación pueden tardar. El proceso utiliza cuatro tareas de compilación para moderar el consumo de RAM. Si se interrumpe, repite el mismo comando: reutiliza los archivos descargados y el directorio de compilación.

Resultado esperado: el listado final incluye `CUDA0` y aparece “Instalacion terminada”. Si falla un paso, el script se detiene y señala el registro correspondiente. No continúes hasta completar la instalación en ambos.

En B, el instalador muestra el usuario, las direcciones de red y la huella de su clave SSH. Conserva estos datos. Ubuntu 22.04 utiliza el servicio `ssh`.

4. Descargar el modelo únicamente en A

```
cd "$HOME/prueba_dos_portatiles"
bash ./02_descargar_modelo_A.sh
```

La descarga es de **9.001.752.960 bytes**, aproximadamente 9 GB decimales. El archivo final queda en `~/cluster_llm/modelos/Qwen3-14B-Q4_K_M.gguf`. Al terminar, el script comprueba su tamaño y SHA-256. Si la conexión se corta, repite el comando para reanudar la descarga.

B recibirá del coordinador los pesos que necesita y podrá conservarlos en una caché local. No descargues una segunda copia completa del GGUF en B.

5. Comprobar las direcciones Ethernet

En ambos portátiles, ejecuta:

```
ip -4 -br address
```

En el equipo informado, Ethernet es `eno1`. Anota la IPv4 de esa interfaz en A y B, sin el sufijo `/24` u otro prefijo. No utilices la dirección de `wlp...`, `docker0` o `lo`.

En B también puedes recuperar el usuario y la huella SSH:

```
whoami
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Ejemplo ilustrativo: A tiene `192.168.50.10`, B tiene `192.168.50.20` y el usuario de B es `alumno`. Sustituye estos datos por los reales. Los dos equipos deben poder comunicarse por la red cableada del laboratorio.

Si el switch está aislado y no hay servidor DHCP, ejecuta ahora el script 07 siguiendo el apartado 13, después de terminar las descargas. Después vuelve al paso 6. Un switch por sí solo normalmente no asigna direcciones IP.

6. Arrancar la GPU remota en B

En una terminal de B:

```
cd "$HOME/prueba_dos_portatiles"
bash ./03_iniciar_rpc_B.sh
```

Resultado esperado: aparecen la RTX 3060, `CUDA0` y el endpoint `127.0.0.1:50052`. El programa queda esperando. Deja la terminal abierta.

RPC escucha exclusivamente dentro de B. La comunicación desde A llegará mediante SSH. No abras el puerto 50052 en el cortafuegos ni cambies la dirección a `0.0.0.0` para esta prueba. RPC sigue siendo experimental; utiliza el entorno de laboratorio y consultas de prueba.

7. Abrir el túnel desde A hacia B

En la primera terminal de A, usa el usuario y la IPv4 Ethernet reales de B. Por ejemplo:

```
cd "$HOME/prueba_dos_portatiles"
bash ./04_abrir_tunel_A.sh alumno@192.168.50.20
```

La primera vez SSH mostrará una huella. Compárala con la que muestra B; si coincide, acepta escribiendo `yes`. Introduce la contraseña del usuario de B. Es normal que no se vea nada al escribirla.

El script comprueba la versión de llama.cpp de B y después abre el túnel; puede pedir la contraseña una segunda vez. Cuando quede abierto puede no mostrar más texto. **Mantén esta terminal abierta y utiliza otra para el paso siguiente.**

El tráfico sigue este recorrido: A → SSH por Ethernet → B → servicio RPC local de B. El cifrado de SSH añade cierto trabajo; las mediciones corresponden a este montaje concreto.

Si SSH no conecta, en B comprueba `sudo systemctl status ssh`. Si el servicio está parado, ejecuta `sudo systemctl start ssh`.

Si el cortafuegos UFW de B está activo y bloquea SSH, permite únicamente la IP de A. Ejemplo, sustituyendo la dirección por la real:

```
sudo ufw status
sudo ufw allow from 192.168.50.10 to any port 22 proto tcp
```

La regla solo es necesaria si el cortafuegos está bloqueando el acceso. No desactives el cortafuegos. Si hay filtrado de red del centro, debe permitir SSH de A a B.

8. Arrancar el modelo en A

En una segunda terminal de A:

```
cd "$HOME/prueba_dos_portatiles"
bash ./05_iniciar_modelo_A.sh
```

El script exige que aparezcan la GPU CUDA local y el dispositivo RPC remoto antes de cargar el modelo. Usa 4096 tokens de contexto, una conversación y reparto 1:1 por capas. Solicita que todas las capas que se pueden descargar a GPU se repartan entre ambas; desactiva el ajuste automático para que un cambio silencioso a CPU no altere la prueba.

Espera a que termine la carga y el servidor esté listo. La primera carga tiene que transferir pesos a B y puede tardar varios minutos. Durante ese periodo la página puede mostrar que el modelo se está

cargando. Mantén también abiertas las terminales de los pasos 6 y 7.

Abre en el navegador de **A**:

```
http://127.0.0.1:8080
```

Escribe una consulta breve en español. El razonamiento prolongado se desactiva para esta prueba inicial. El chat escucha solo dentro de A; esa misma dirección en B no abre el chat de A.

9. Confirmar que participan las dos GPU

En una terminal adicional de **cada portátil**, ejecuta:

```
watch -n 1 nvidia-smi
```

Al terminar la carga debes ver varios GB de VRAM ocupados en ambas GPU. Mientras se genera una respuesta puede aumentar la actividad en momentos distintos: el reparto por capas implica dependencias entre los cálculos. No es necesario que ambas indiquen 100 % de uso simultáneamente.

Revisa también el registro de arranque de A: debe mencionar buffers del modelo en CUDA local y RPC, y las capas descargadas a GPU. La presencia de algún buffer CPU o memoria mapeada, por sí sola, no significa que toda la inferencia se esté ejecutando en CPU.

Para abandonar `watch`, pulsa Ctrl+C; esto no detiene el modelo.

10. Ejecutar una consulta con medición

En una tercera terminal de A, cuando el servidor esté listo:

```
cd "$HOME/prueba_dos_portatiles"
bash ./06_probar_A.sh
```

Envía una consulta breve, muestra la respuesta y guarda un JSON en `~/cluster_llm/resultados`. Recoge:

- Tiempo desde el envío hasta el primer texto recibido por el cliente.

- Tiempo total de la consulta, sin incluir el arranque del modelo.

- Tokens por segundo comunicados por el servidor, si están presentes.

- Texto de la respuesta, uso de tokens y motivo de finalización.

La salida está limitada a 256 tokens. Si alcanza ese límite, se indica en pantalla. Una consulta breve comprueba el funcionamiento; para comparar rendimiento conviene repetirla tres veces con la misma configuración y conservar todos los resultados. No uses el chat simultáneamente mientras mides.

La primera carga del modelo, el tiempo hasta el primer texto y la velocidad de generación son medidas distintas. Un resultado favorable en una consulta corta no garantiza lo mismo con documentos largos o varios usuarios.

11. Detener y volver a arrancar

Detén en este orden, pulsando Ctrl+C en la terminal correspondiente:

1. **A:** servidor del modelo, paso 05.

2. **A:** túnel SSH, paso 04.

3. **B:** servidor RPC, paso 03.

Si utilizaste las IP fijas del apartado 13, ejecuta ahora el script 08 en cada portátil para volver a DHCP. Hazlo desde una terminal local, después de detener los tres procesos anteriores.

Para volver a utilizarlo otro día, conecta ambos portátiles al switch aislado, repite 07 en cada uno y después los pasos 03, 04 y 05, en ese orden. Repite 06 si quieres otra medición. La instalación y la descarga se conservan. Si utilizas una red que ya asigna IP por DHCP, basta con comprobar las direcciones y arrancar 03, 04 y 05.

El servicio SSH instalado en B puede seguir activo. Si se instaló únicamente para esta prueba y nadie más lo utiliza, puedes pararlo con `sudo systemctl stop ssh`. Para la siguiente sesión arráncalo con `sudo systemctl start ssh`. No pares SSH si el centro lo utiliza para administrar ese equipo.

Si añadiste la regla UFW de ejemplo y ya no la necesitas, elimínala con la misma IP que utilizaste:

```
sudo ufw delete allow from 192.168.50.10 to any port 22 proto tcp
```

12. Problemas habituales

Mensaje o síntoma	Qué hacer
`nvidia-smi` falla	Resolver primero el controlador NVIDIA. No ejecutar la instalación con una GPU que no se detecta.
El segundo equipo no tiene una RTX de capacidad CUDA 8.6	Detenerse y revisar su informe. Este paquete está compilado para el hardware identificado.
Error al instalar paquetes	Revisar Internet, espacio y repositorios. Conservar el registro; no cambiar controladores ni repositorios al azar.
Compilación finalizada por falta de memoria	Cerrar aplicaciones y repetir. Se puede cambiar `--parallel 4` por `--parallel 2` en el instalador.
`Permission denied` al conectar por SSH	Comprobar usuario y contraseña de B. Si el centro exige claves SSH, usar las credenciales permitidas por su configuración.
SSH muestra que ha cambiado la clave del equipo	Verificar la IP y la huella en B. No desactivar la comprobación de claves.
`Connection refused` en RPC	Mantener abierto 03 en B y 04 en A. Revisar que 03 no haya terminado con error.
Puerto 50052 o 8080 ocupado	Cerrar la terminal de la ejecución anterior o identificar el proceso con `ss -ltnp`. Los scripts no matan procesos automáticamente.
No aparece la GPU RPC	Comprobar túnel, servicio de B y misma versión. Conservar las salidas de ambos equipos.
Menos de 5000 MiB de VRAM libres	Cerrar otras aplicaciones que utilicen NVIDIA. Consultar los procesos en `nvidia-smi`.
`out of memory` al cargar	Detener A, reiniciar RPC en B si se cerró, y volver a iniciar A con contexto reducido como se indica debajo.
El modelo carga pero va lento	Medir primero; revisar uso de GPU, conexión Gigabit, temperatura, cargadores y aplicaciones concurrentes.
Huella SHA-256 incorrecta	Conservar el archivo para revisar. Si repites desde cero, renombra solo ese GGUF o su `.part`; no borres carpetas del proyecto.
Una descarga parcial no se puede reanudar	Renombrar el `.part` afectado y repetir 02 para iniciar una descarga nueva, con espacio suficiente.
El script 07 no encuentra `eno1`	Consultar `nmcli device status` y pasar el nombre de la interfaz Ethernet como segundo argumento, después de A o B.
La subred coincide con otra conexión	Desconectar temporalmente la Wi-Fi o VPN indicada. No modificar Docker ni otras redes sin identificar el conflicto.
El script 08 deja DHCP pendiente	Conectar a una red con servidor DHCP y repetir 08. Un switch aislado normalmente no ofrece ese servicio.
El script 08 no encuentra la configuración guardada	Usar el mismo usuario y la misma interfaz que en 07. Conservar `~/cluster_llm/red`; el script no modifica perfiles sin ese registro.
`$'\r'` o errores al principio del script	Volver a extraer el ZIP original. Los archivos se entregan con saltos de línea Linux; algunos editores de Windows pueden cambiarlos.

Para reducir la memoria de contexto, detén el modelo de A y ejecuta:

```
bash ./05_iniciar_modelo_A.sh 2048
```

Si sigue fallando con ambas GPU libres, guarda los registros antes de modificar el reparto o enviar capas a CPU: ese cambio alteraría lo que estamos midiendo.

13. Switch aislado sin servidor DHCP

Este apartado utiliza los scripts 07 y 08 incluidos en el paquete. Termina antes la instalación y la descarga con Internet. Ambos scripts funcionan sin Internet y utilizan NetworkManager, presente normalmente en Ubuntu Desktop, y Python 3.

Preparar el cableado: conecta los dos portátiles a un switch encendido y separado de la red del centro. Desactiva temporalmente Wi-Fi y VPN desde Ubuntu si quieres que los equipos queden aislados también por esas conexiones. Los scripts actúan únicamente sobre la interfaz Ethernet indicada.

Ejecuta los comandos desde el teclado de cada portátil, con tu usuario habitual y sin `sudo` delante. El script pedirá la contraseña cuando la necesite. Rechaza su ejecución dentro de una sesión SSH para evitar cortar esa sesión.

En A:

```
cd "$HOME/prueba_dos_portatiles"
bash ./07_configurar_red_aislada.sh A
```

En B:

```
cd "$HOME/prueba_dos_portatiles"
bash ./07_configurar_red_aislada.sh B
```

Por defecto usan `eno1`, la interfaz del equipo informado. Si el segundo portátil tiene otro nombre, consúltalo con `nmcli device status` y añade ese nombre. Ejemplo: `bash ./07_configurar_red_aislada.sh B enp3s0`. Debe ser una interfaz física de tipo Ethernet administrada por NetworkManager.

La configuración queda así:

Campo	Portátil A	Portátil B
Dirección	`192.168.50.10`	`192.168.50.20`
Máscara	`255.255.255.0`	`255.255.255.0`
Puerta de enlace	Vacía	Vacía
DNS	Vacío	Vacío

El script crea un perfil de prueba separado y conserva los perfiles anteriores en NetworkManager. Guarda la referencia del perfil que estaba activo y el estado de la operación en `~/cluster_llm/red/eno1.json` (el nombre cambia si utilizas otra interfaz). No borres esa carpeta ni la copies de un portátil al otro. El registro no es una copia completa de los perfiles: estos siguen guardados en NetworkManager.

Antes de aplicar la IP, comprueba conflictos con subredes de otras interfaces y solicita a NetworkManager detectar una dirección duplicada en el cable. Estas comprobaciones no garantizan que el switch esté físicamente aislado: asegúrate del cableado. Si falla la activación, intenta recuperar el perfil que estaba activo.

Comprobar la comunicación desde A:

```
ping -c 3 192.168.50.20
```

Si no responde, comprueba cable, IP y cortafuegos; algunos cortafuegos pueden filtrar ping aunque SSH funcione. Continúa en el paso 6 y utiliza `192.168.50.20` para B. El usuario de SSH será el usuario real de B.

Volver a DHCP al terminar, en ambos portátiles:

1. Detén modelo, túnel y RPC como indica el apartado 11.

2. Conecta el cable Ethernet a la red habitual que disponga de DHCP.

3. Ejecuta el siguiente comando en una terminal local de cada portátil:

```
cd "$HOME/prueba_dos_portatiles"
bash ./08_restaurar_dhcp.sh
```

Si utilizaste otra interfaz, indícala también aquí: `bash ./08_restaurar_dhcp.sh enp3s0`.

El script retira el perfil de IP fija de la prueba y vuelve a activar el perfil anterior si usaba DHCP. Si antes utilizabas una IP manual o no había perfil activo, crea un perfil DHCP independiente y conserva el anterior. Una red con autenticación especial puede requerir seleccionar el perfil habitual desde Configuración → Red; el script no configura credenciales del centro.

Si ejecutas 08 todavía en el switch aislado: retira el perfil fijo y prepara DHCP, pero probablemente no obtendrá dirección. Puede tardar hasta 45 segundos y terminar con un aviso de DHCP pendiente (código de salida 2). Conecta después a la red habitual y repite exactamente el comando 08. El mensaje “Conexión DHCP activa” solo aparece cuando la activación termina y hay una IPv4 en la interfaz; no acredita acceso a Internet.

Puedes repetir 07 con el mismo rol y 08 sin acumular perfiles de prueba. Tras reiniciar un portátil, vuelve a ejecutar 07 para activar sus IP de laboratorio: ese perfil tiene la conexión automática desactivada. Al terminar, vuelve a activar Wi-Fi o VPN si las habías desactivado.

14. Qué resultados conservar del laboratorio

Copia desde **A** la carpeta `~/cluster_llm/resultados`, el último registro `modelo_A_...log` y `~/cluster_llm/entorno_A.txt`. Desde **B**, copia el último `rpc_B_...log` y `~/cluster_llm/entorno_B.txt`. Los registros están en `~/cluster_llm/logs`.

Anota en la ficha de resultados si utilizaste contexto 4096 o 2048 y si había otros programas abiertos. Los registros pueden contener consultas: para esta primera práctica utiliza texto de ejemplo.

Referencias y alcance de la comprobación

- Versión fijada de llama.cpp.

- RPC en la revisión utilizada.

- Opciones de llama-server en esa revisión.

- CUDA 12.8 para Linux.

- Modelo oficial y revisión fijada.

- NetworkManager: gestión de perfiles con nmcli.

- NetworkManager: IP manual, DHCP y detección de direcciones duplicadas.

La preparación se basa en el informe del portátil de referencia y en documentación y código oficiales. Las comprobaciones locales del paquete se describen en `COMPROBACIONES.txt`. La instalación real, el reparto en las RTX 3060 y el rendimiento por Ethernet se comprobarán al ejecutar estos pasos en el centro.

7. Validación y registro de resultados

Evidencias disponibles

Comprobación	Estado
Diagnóstico de un portátil del centro	Completado, tres informes del mismo equipo
Diagnóstico del segundo portátil	Pendiente
Sintaxis Bash y Python del paquete	Comprobada en el entorno de preparación
Lectura SSE del cliente de medición	Simulada, respuesta completa y truncada
Lógica de red	12 pruebas simuladas superadas
IP fija y DHCP en Ubuntu real	Pendiente
Compilación CUDA en los equipos	Pendiente
Descarga y hash del archivo real	Pendiente en A
Carga distribuida, VRAM y velocidad	Pendiente

Las simulaciones bloquean la ejecución de procesos externos y sustituyen NetworkManager y datos del sistema. Cubren IP de A/B, repetición, cambio de rol, recuperación de fallo, DHCP previo, perfil previo manual, DHCP ausente y reintento, perfil alterado, falta de estado, nombre alterado y rechazo en Windows. No reproducen el comportamiento completo de Ubuntu ni de NetworkManager.

Para repetir estas pruebas desde la raíz del repositorio, con Python 3:

```
python3 tests/test_red_simulada.py
```

Qué debe demostrar la sesión en el centro

1. Los dos equipos cumplen los requisitos y compilan la misma revisión.
2. A y B se comunican por Ethernet con las IP previstas.
3. A detecta su CUDA local y la GPU RPC remota.
4. El registro de carga menciona memoria de modelo en ambas GPU.
5. La consulta devuelve una respuesta y se guarda un resultado JSON.
6. La prueba se repite sin modificar contexto, modelo ni condiciones.
7. Se detienen los servicios de prueba y se recupera DHCP.

Ficha para copiar en una nueva sesión

```
Fecha y responsable:
Commit de este repositorio:
Modelo, revisión y SHA-256:
Revisión de llama.cpp:
GPU / VRAM libre antes del arranque en A y B:
RAM y versión Ubuntu en A y B:
Controlador NVIDIA / CUDA Toolkit:
Ethernet y velocidad negociada en A y B:
Contexto: 4096 / 2048
Caché RPC: primera carga / reutilizada
Tiempo de carga del modelo (medido aparte):
VRAM ocupada en A / B tras cargar:
Consulta 1: primer texto / tiempo total / tokens por segundo:
Consulta 2: primer texto / tiempo total / tokens por segundo:
Consulta 3: primer texto / tiempo total / tokens por segundo:
Otros programas, temperatura, incidencias:
IP fija y vuelta a DHCP: resultado en A / B:
Conclusión y siguiente cambio propuesto:
```

Guarda `~/cluster_llm/resultados`, los registros de modelo de A y RPC de B, y `entorno_A.txt` / `entorno_B.txt`. Revisa consultas e identificadores antes de compartirlos. No rellenes campos pendientes con estimaciones.

Compara tiempo de carga, espera hasta el primer texto y velocidad de generación por separado. Si se quiere evaluar una mejora de velocidad frente a un solo equipo, primero hay que definir una configuración de referencia que realmente pueda ejecutarse en ese equipo y documentar cualquier uso de CPU.

8. Adaptación a otros equipos y ampliación a cuatro

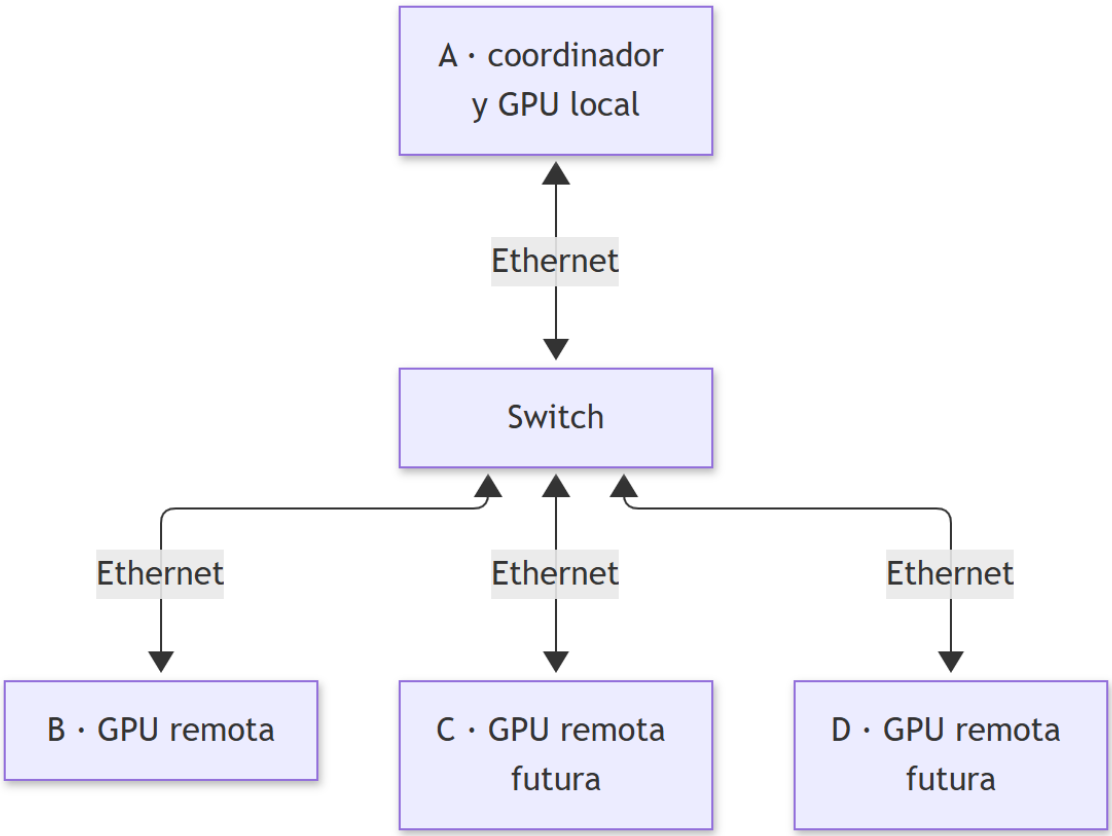
Lo que se puede reutilizar

El procedimiento de diagnóstico, la separación entre instalación y uso aislado, el registro de resultados y la organización coordinador/remotos sirven como base para otros laboratorios. Los scripts actuales tienen requisitos concretos: Ubuntu 22.04 x86_64, NVIDIA con capacidad CUDA 8.6, toolkit 12.8, un rol A y un rol B.

Antes de adaptar el paquete, recopila datos de cada nodo. Revisa sistema, controlador, arquitectura CUDA, VRAM libre, RAM, espacio, interfaz Ethernet y velocidad del enlace. No elimines las comprobaciones del instalador para forzarlo sobre hardware distinto: cambia y documenta los parámetros compatibles, y vuelve a comprobar la configuración.

Cambio de entorno	Aspectos que hay que revisar
Otra GPU NVIDIA	Arquitectura de compilación, toolkit y controlador compatibles, memoria y reparto
Otra versión de Ubuntu	Repositorios, paquetes, compilador y funcionamiento de NetworkManager
Otra interfaz Ethernet	Argumento de interfaz en 07 y 08
Menos VRAM	Tamaño del modelo, cuantización, contexto y buffers
GPU de otro fabricante	Backend y compilación; este instalador CUDA no la cubre
Más usuarios	Memoria y concurrencia; la configuración actual es de una conversación

Fase futura con cuatro equipos



Este esquema representa el objetivo posterior, **no una configuración ya implementada**. Los scripts actuales solo aceptan A y B y el servidor de A selecciona una GPU RPC. No basta con ejecutar los mismos scripts en cuatro equipos con el rol B: se repetirían direcciones y faltaría definir los otros endpoints.

Pasos de la ampliación

1. Completar y conservar una sesión válida con dos nodos.

2. Recopilar informes independientes de C y D.

3. Reservar direcciones diferentes; propuesta: A `.10`, B `.20`, C `.30`, D `.40` en la misma subred `192.168.50.0/24`.

4. Ampliar los roles de instalación y los perfiles de red, incluido el retorno a DHCP de cada equipo.

5. Ejecutar un servidor RPC local en cada remoto y crear tres túneles desde A, con puertos locales diferentes, por ejemplo 50052, 50053 y 50054. Cada túnel puede llegar al 50052 local del remoto correspondiente.

6. Enumerar los dispositivos de la revisión fijada y adaptar `--rpc`, `--device` y `--tensor-split` a los dispositivos realmente detectados. No asumir nombres sin consultarlos.

7. Empezar con el mismo modelo y contexto para observar el efecto de añadir nodos. Después evaluar otro modelo si hay capacidad.

8. Comparar memoria, carga, latencia, velocidad y estabilidad con la sesión de dos equipos.

Cuatro RTX iguales sumarían 24 GiB nominales distribuidos. La conexión de A y las transferencias entre nodos pueden limitar el rendimiento. Un switch más rápido no mejora adaptadores de 1 Gb/s; cualquier cambio de red debe comprobarse de extremo a extremo. La decisión de ampliar dependerá de capacidad útil y experiencia real, no del número de GPU por sí solo.

9. Fuentes y versiones

Material del proyecto

- Conversación de diseño: necesidad de modelos locales, diagnóstico, prototipo, uso aislado y retorno a DHCP.

- Tres informes del mismo portátil, fechados el 23 de octubre de 2026. Se publica su síntesis técnica, conservando los originales fuera del repositorio.

- Artículo Word aportado como referencia: “How to run ONE LLM model across different machines”. No se redistribuye su contenido ni se le atribuyen pruebas de este hardware.

- Scripts, guía y comprobaciones conservados en `prueba_dos_portatiles`.

Referencias primarias

- llama.cpp v0.7.0.

- RPC en la revisión elegida.

- Servidor de llama.cpp en esa revisión.

- CUDA 12.8 para Linux.

- Qwen3-14B GGUF en la revisión elegida.

- NetworkManager nmcli.

- NetworkManager IPv4.

Las referencias de llama.cpp y del modelo están fijadas a revisiones. Las de NetworkManager describen su documentación actual: la ejecución real debe comprobarse en Ubuntu 22.04. Las dependencias descargadas conservan sus licencias y condiciones originales.

Reproducibilidad

El archivo `prueba_dos_portatiles/versiones.env` es la referencia de versión y hash del modelo. Guarda también el commit de este repositorio en cada sesión. Actualizar llama.cpp en un solo equipo puede romper la correspondencia de RPC; cualquier actualización debe probarse en todos los nodos y acompañarse de nuevos resultados.

10. Comprobaciones del paquete

COMPROBACIONES DEL PAQUETE

Fecha de preparación: 23 de octubre de 2026.

Comprobaciones realizadas en la preparacion inicial, antes de incorporar la red:
- Sintaxis Bash de los siete scripts y versiones.env con bash -n.
- Sintaxis Python del cliente de consulta.
- Lectura de una respuesta SSE simulada, captura de texto y tiempos y guardado JSON.
- Rechazo de una respuesta SSE truncada.
- Correspondencia del tag v0.7.0 con la revisión fijada de llama.cpp mediante la API oficial.
- Tamaño y SHA-256 del GGUF según su referencia LFS oficial, sin descargar los 9 GB.
- Opciones principales del servidor y de RPC contrastadas con la revisión fijada.
- Entrega con UTF-8 sin BOM y saltos de línea LF.

Ampliacion de red del 23 de octubre de 2026:
- Incorporados 07_configurar_red_aislada.sh, 08_restaurar_dhcp.sh y red_cluster.py.
- Utilizan perfiles de NetworkManager separados y conservan los originales.
- Guia Markdown y HTML actualizadas; paquete ZIP regenerado.
- Revision posterior solicitada: 12 pruebas de red simulada superadas.
- NetworkManager y sysfs simulados; bloqueo de procesos externos en esas pruebas.
- Casos: IP de A/B, repeticion, cambio de rol, fallo de activacion y recuperacion,
DHCP anterior, perfil anterior manual, ausencia de DHCP y reintento,
perfil alterado, ausencia de registro, nombre alterado y rechazo en Windows.
- Sintaxis Bash de todos los scripts y sintaxis de ambos archivos Python comprobadas.
- Corregidos: aviso de interfaz sin enlace, recuperacion ante errores de E/S
durante activacion y rechazo de gateway/DNS/IPv6 alterados en el perfil de prueba.
- Las pruebas simuladas no ejecutan nmcli, sudo ni cambios de red reales.

Pendiente en los portátiles del centro:
- Instalación de dependencias y compilación con CUDA 12.8 en Ubuntu 22.04.
- Descarga de los pesos y comprobación de su contenido real.
- Carga del modelo en las dos RTX 3060 de 6 GB.
- Comunicación SSH/RPC real por Ethernet, consumo de VRAM y velocidad.
- Activacion de IP fija y restauracion de DHCP con NetworkManager en los dos equipos.

No se han instalado CUDA ni servicios de Linux en el portátil personal.
No se ha modificado la configuracion de red del portatil personal.
Las pruebas simuladas del cliente no acreditan rendimiento ni compatibilidad completa del clúster.

11. Recopilación de datos de hardware

RECOPILAR DATOS DE LOS PORTATILES DEL CENTRO

1. Copia recopilar_datos_cluster.sh a un pendrive y llevalo al centro.

2. En cada portatil Ubuntu, copia el script a una carpeta local, por ejemplo Descargas. Conecta el cable Ethernet al switch que se utilizara en la prueba. Si usas un adaptador Ethernet USB, conectalo tambien.

3. Abre la carpeta que contiene el script. Pulsa con el boton derecho sobre un espacio vacio y elige "Abrir en una terminal".

4. Ejecuta:

bash ./recopilar_datos_cluster.sh

No hace falta marcar el archivo como ejecutable ni ejecutar el script con sudo.

5. El script mostrara la ruta del informe generado. Lo encontraras en la carpeta informes_cluster, dentro de tu Carpeta personal. Cada archivo incorpora el nombre del equipo, la fecha y un identificador para evitar sobrescrituras.

6. Si avisa de que falta ethtool, puedes completar los datos de Ethernet ejecutando estos comandos con conexion a Internet:

sudo apt update && sudo apt install ethtool
bash ./recopilar_datos_cluster.sh

Conserva el informe mas reciente de ese equipo. Si no puedes instalar ethtool, entrega el informe disponible: incluye la velocidad negociada cuando Linux consigue consultarla, aunque falten las velocidades admitidas.

7. Repite en cada portatil. Copia los informes .txt al pendrive y conservalos en el archivo privado del proyecto. Compara los datos con docs/02_hardware.md del repositorio. Antes de compartirlos, elimina identificadores personales y de red. Publica solo una sintesis tecnica de cada equipo.

El script consulta CPU, RAM, GPU, controlador NVIDIA y conexiones Ethernet. Solo crea la carpeta de informes y el archivo de texto; no instala programas. Si nvidia-smi falta o falla, el informe lo registra: entrega igualmente el archivo. La velocidad de Ethernet solo se puede interpretar con un enlace activo.

12. Versiones y huella del modelo

```
# Versiones fijadas para esta prueba. Copiar el mismo paquete a A y B.
LLAMA_TAG='v0.7.0'
LLAMA_COMMIT='620d2225cc436ab0b001e3200e590e3b286ef9ab'
MODEL_REPO='Qwen/Qwen3-14B-GGUF'
MODEL_REVISION='530227a7d994db8eca5ab5ced2fb692b614357fd'
MODEL_NAME='Qwen3-14B-Q4_K_M.gguf'
MODEL_SHA256='500a8806e85ee9c83f3ae08420295592451379b4f8cf2d0f41c15dffeb6b81f0'
MODEL_BYTES='9001752960'
```